

Regular expression

CS 22B, Spring 2026

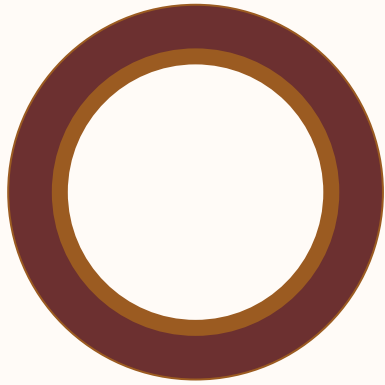
Module 02: Control flow

Presented by Jessica Huynh-Westfall, Ph.D.

AGENDA



Regular expression



Chapter 16: Regular expression

The Quick Python Book

Or Python for Biologist Chap 7



What is regular expression?

04

```
my_str = "For with knowledge comes pain. To limit the pain  
I limit my answers."  
if re.search("knowledge comes pain", my_str):  
    print("quote found")
```

What if searching for pattern that has special characters (eg., \n or \t)?

```
my_str2 = "Knowledge comes pain.\nLimit the pain."  
if re.search(r"pain.\nLimit", my_str2):  
    print("quote found")
```

Using operations in regular expression

05

Using pipe character, |, to select alternative characters

```
GG(A|T)CC can select for A or T
```

Use a pair of square bracket, [] with list of characters inside that represent characters that can match pattern

```
GG[A-Z]CC can select for A or T
```

Use a caret, ^, at start of a character group to negate it and match any character not in the group

```
GG[^CG]CC will not select C or G
```

Search for pattern in a string

06

Regular expression is a way to recognize and extract data from certain patterns of text.

- Recognize a string (text) that is a match to text or string

```
import re
regexp = re.compile("Good luck, babe")
file = open("goodluck_babe.txt", "r")
count = 0
for line in file.readlines():
    if regexp.search(line):
        count = count + 1
file.close()
print(count)
```

re.Pattern is text pattern

Read in each line and search for pattern. If pattern found, count +1 increment

Class exercise

07

```
import re
regexp = re.compile("Good luck, babe")
file = open("goodluck_babe.txt", "r")
count = 0
for line in file.readlines():
    if regexp.search(line):
        count = count + 1
file.close()
print(count)
```

CL 6.1

There are 12 instances of the string good luck, babe but our code only counted 6. How can we change the regex so that it can capture all 12 instances?

Use quantifier to describe variation in count

08

Quantifier	Description	Pattern	Possible matches
+	Followed by character must be present but can be repeat <i>one or more times</i>	GGGA+TTT	GGGATTT, GGGATT, GGGGATT, etc.
*	Followed by character means character is optional and can be present <i>0 or more times</i>	GGGA*TTT	GGGATTT, GGGATTTTTTTTTT, etc.
{#}	Following a character with number inside { } will match exactly number of repeats	GA{5}T	GAAAAAT but not GAAAAT or GAAAAAAT.
{#, #}	Following a character with a pair of number inside { } will allow for acceptable range of number of repeats	GA{2,4}T	GAAT, GAAAT and GAAAAT but not GAT or GAAAAAT

Use of position in string

09

Position	Description	Pattern	Possible matches
^	Matches the start of a string	^AAA	AAATTT but not GGGAAATTT
\$	Matches the end of a string	GGG\$	AAAGGG but not AAAGGGCCC

Class exercise

10

CL 6.2

Explain the following complex pattern

`^ATG[ATGC]{30,1000}A{5,10}$`

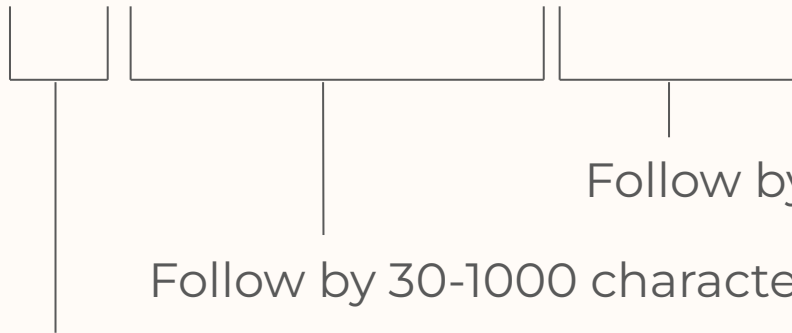
Reading the pattern from left to right, what will it match?

Class exercise - Answer

CL 6.2

Explain the following complex pattern

`^ATG[ATGC]{30,1000}A{5,10}$`



Start with 'ATG'

Follow by 30-1000 characters which can be A,T,G,C

Follow by tail of 5-10 As

Extracting substring that matched

- Matched pattern is return as object that is evaluated as true.
- Matched object represents result of regular expression search
- Matched object has methods associated with it to work with data

```
seq = "ATGACGTACGTACGACTG"
```

```
matching = re.search(r"GA[ATGC]{3}AC", seq)  
print(matching.group())
```

Class exercise

Extracting pattern with parentheses - Capturing groups

CL 6.3

What is captured in group(), group(1) and group(2)?

```
seq = "ATGACGTACGTACGACTG"

matching = re.search(r"GA([ATGC]{3})AC([ATGC]{2})AC", seq)
print("entire match: " + matching.group())
print("first bit: " + matching.group(1))
print("second bit: " + matching.group(2))
```

Split string using regular expression

14

Split function takes regular expression pattern as argument

```
dna = "ACTNGCATRGCTACGTYACGATSCGAWTCG"  
runs = re.split(r"[^ATGC]", dna)
```

Find position of individual groups from the start and end position so the whole pattern

```
print("group one start: " + str(matching.start(1)))  
print("group one end: " + str(matching.end(1)))
```

Finding multiple matches

15

`re.findall` returns a list of all matches of a pattern in a string.

```
dna = "ACTGCATTATATCGTACGAAATTATACGCGCG"  
runs = re.findall(r"[AT]{4,100}", dna)
```

`re.finditer` returns a sequence of match objects. To use object, need to implement in loop to return value

```
runs = re.finditer(r"[AT]{3,100}", dna)  
for match in runs:  
    print(m)
```

Recap

16

Goal is to write a reg expression containing the letters d and e in that order

Expression	Meaning
d.*e	"d" followed by anything (including nothing) until the first "e"
d.e	"d" followed by exactly one character before "e"
d.+e	"d" followed by at least one character before "e"
de	Exact substring "de"
d*e	Zero or more "d" before "e"

Class exercise

17

```
accession = ["xkn59438", "yhdck2", "eihd39d9", "chdsye847", "hedle3455",  
"xjhd53e", "45da", "de37dp"]
```

CL 6.4

Write a program that will print only the accession names that satisfy the following criteria – treat each criterion separately:

- A. contain the letters d and e in that order
- B. contain both the letters d and e in any order
- C. start with x or y and end with e
- D. contain three or more numbers in a row
- E. end with d followed by either a, r or p